

Smart Data Retrieval: AD-Adoption Benchmark Report

Date: 25 June, 2026

Subject: Evaluation of Production Search Architecture (Hashing vs. Hybrid Retrieval)

1. Executive Summary

This report details the performance and accuracy benchmarks comparing our legacy **Hashing Retriever** against the proposed **Hybrid Retriever** (Fast Bi-Encoder + BM25).

The goal of this benchmark was to prove that upgrading our retrieval engine would solve our current "missed search" issues without violating strict production latency and memory limits.

The Verdict: The Hybrid Retriever successfully passed all AD-Adoption criteria. By utilizing a highly optimized, lightweight embedding model (bge-small-en-v1.5) alongside mathematical score fusion, we achieved a **massive increase in search accuracy** while maintaining a **sub-100ms query latency**.

2. Benchmark Methodology

To ensure a strictly fair, "apples-to-apples" comparison, the evaluation harness was built to mimic our exact production environment:

- **Dataset:** 4,922 document chunks loaded from `small_corpus.jsonl`.
- **Golden Evaluation Set:** 20 highly complex, paraphrase-heavy queries mapped to definitive ground-truth document chunks.
- **Data Formatting:** Both algorithms were restricted to searching *only* the summary and keywords of the documents, perfectly matching the constraints of our current `document_chunking_service.py`.
- **Hardware Profile:** Standard CPU execution (no specialized GPUs required).

3. Key Findings

A. Accuracy & Relevance (The "Smart" Upgrade)

The legacy Hashing system relies entirely on exact keyword matching. When a user queries concepts using different vocabulary than the document, the system fails entirely.

The Hybrid Retriever leverages semantic understanding to bridge this gap.

Example Win (Query 19): * Query: "What act established requirements for legislation that imposes duties on state or local governments without funding?"

- **Target:** "Unfunded Mandates Reform Act of 1995"
- **Hashing Result:** Missed completely (Recall: 0.0)
- **Hybrid Result:** Found instantly as the #1 result (Recall: 1.0, MRR: 1.0)

For queries where both systems found the document (e.g., Query 1), the Hybrid system consistently ranked the correct document higher, meaning the LLM gets better context to

generate its final answer.

B. Latency & Performance (The Operations Impact)

A major concern heading into this benchmark was that AI-based Semantic Search would be too slow for production workers. By dropping the Cross-Encoder (Reranker) and using a computationally light "Fast Bi-Encoder", we solved this constraint.

Metric	Legacy Hashing	New Hybrid (Fast Bi-Encoder)
Query Latency	~6 - 9 ms	~ 60 - 80 ms
Query Memory Spike	~7.45 MB	~ 0.28 MB

- **Latency Analysis:** While Hybrid is inherently slower than a simple hash lookup, ~60-80ms is well within standard UI/UX budgets for real-time chat applications. The user will not notice this delay.
- **Memory Analysis:** Surprisingly, the Hybrid Retriever causes a significantly *smaller* RAM spike during the actual query (~0.28MB vs ~7.45MB). This is because we pre-normalize our dense vectors into active memory when the app starts. At query time, the system only needs to perform an ultra-fast mathematical dot-product, whereas the legacy system dynamically generates heavy sparse matrices.

4. Architectural Shift

By moving to this architecture, we have built an internal capability that rivals enterprise SaaS products like Omni.

1. **No Reranker Overhead:** We eliminated ~350ms of compute time by relying on mathematical score fusion (70% Semantic, 30% Keyword).
2. **Reduced Code Complexity:** We are replacing thousands of lines of custom vector math with standard, battle-tested LangChain and SentenceTransformer libraries.
3. **Future-Proofing:** If accuracy demands increase, we can seamlessly swap the small model for a large model with a single line of code.

5. Recommendation & Next Steps

Based on these results, **we recommend moving forward with the phased rollout of the Hybrid Retriever.**

Phase 1 Execution Plan (Next Week):

1. Integrate BAAI/bge-small-en-v1.5 alongside our BM25 index into the development environment.
2. Maintain the current chunking logic (summary + keywords) to minimize breaking changes.
3. Hook the new pipeline into LangSmith to begin capturing visual telemetry of the improved search results in a staging environment.
4. Prepare for production shadow testing.